Software Architecture 2024[2026]

Lecture 4, part 2/3

System context

Dimitri Van Landuyt

Wouter Joosen

KU LEUVEN DistriNet

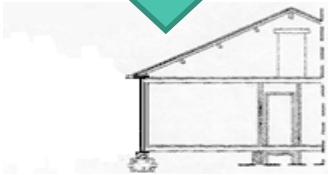
Introduction

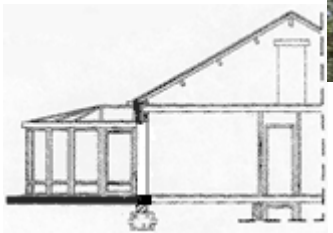
In this stage, we approach the system as a **black box**

- **Functionality** expressed as use cases: **interactions between the system and** actors
- **Non-functional aspects** expressed ASRs and QASs: **Stimuli** (from a **Source** onto an **Artifact** in a specific **Environment**) leading to quality attribute **Response**, externally verifiable by an observer (**Response Measure**)

We generally refrain from opening the black box

From Descriptive to Prescriptive Modelling





Descriptive

Prescriptive

System context

System context

- Context boundary is the distinction/delineation between what is part of the system under design and what is not
  - System boundary
  - *In* vs *out*
- Modeling context boundaries = characterizing the ‘surface’ between the system and its environment
- Proper knowledge of system context is **crucial** to understand
  - the role of the system in its **environment**,
  - existing constraints imposed by the environment

System context

- Context boundary is the distinction/delineation between what is part of the system under design and what is not
  - System boundary
  - *In vs out*
- Modeling context boundaries = characterizing the ‘surface’ between the system and its environment
- Proper knowledge of system context is **crucial** to understand
  - the role of the system in its **environment**,
  - existing constraints imposed by the environment

DOMAIN ANALYSIS
NO GREENFIELD DEVELOPMENT

Sounds trivial?

System context

Key context-related questions

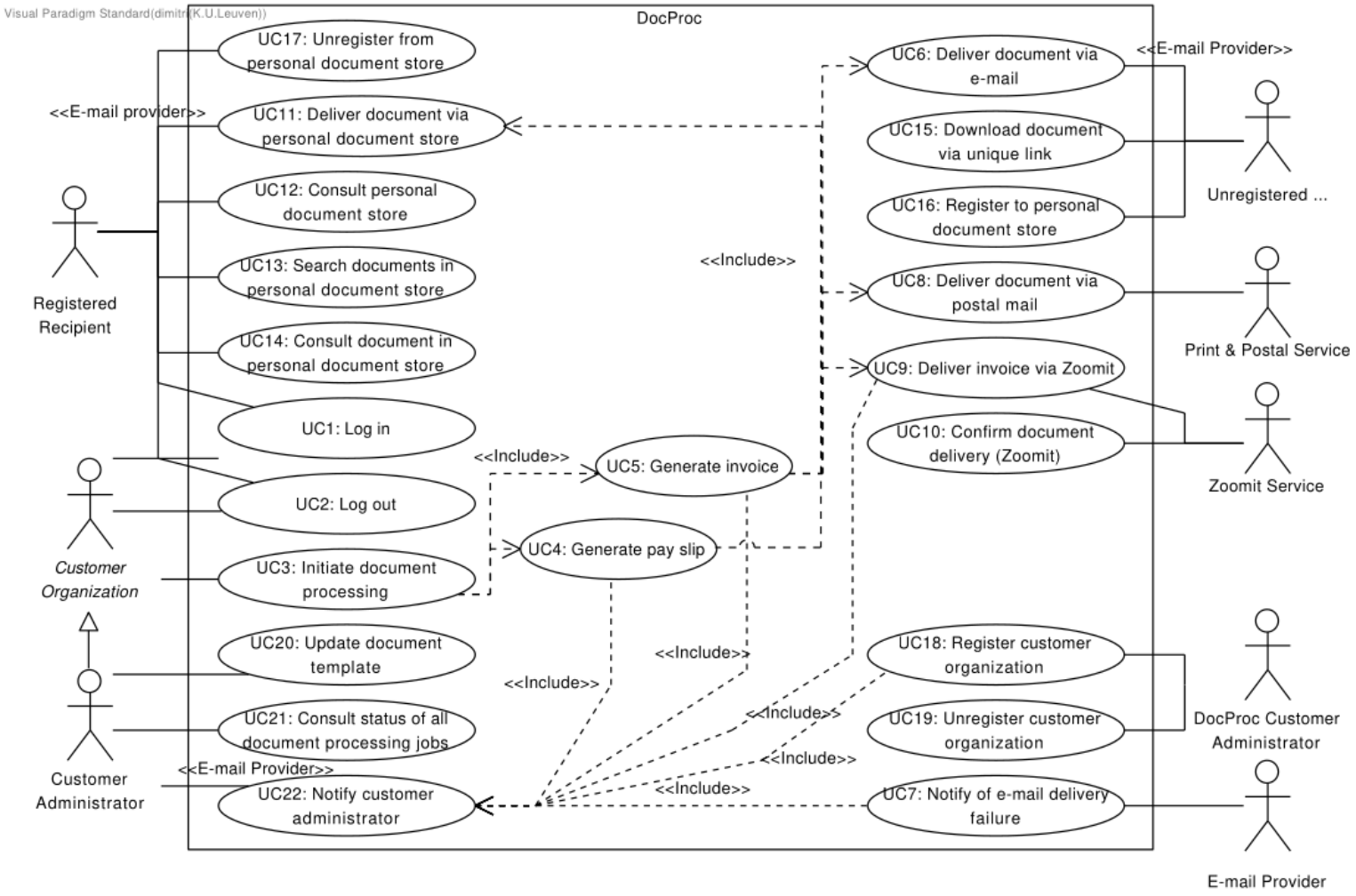
IN **OUT**

- *What software will we develop versus what software will we reuse?*
- *Which services will we provide versus which third-party services will we use?*
- *Which machines will be under our control versus which machines will not be?*
- *What functionality should we support versus what functionality is already there?*
- *How do we interface with the outside world and how does the outside world talk to us?*

Many of these answers are **not** up to you

Context during requirements elicitation

- How does context affect functional and non-functional requirements?


Visual Paradigm Standard(dimitri(K.U.Leuven))
DocProc
UC17: Unregister from personal document store
<<E-mail provider>>
UC11: Deliver document via personal document store
UC12: Consult personal document store
UC13: Search documents in personal document store
UC14: Consult document in personal document store
UC1: Log in
UC2: Log out
UC3: Initiate document processing
UC20: Update document template
UC21: Consult status of all document processing jobs
<<E-mail Provider>>
UC22: Notify customer administrator
Registered Recipient
Customer Organization
Customer Administrator
<<Include>>
UC5: Generate invoice
UC4: Generate pay slip
<<Include>>
<<Include>>
<<Include>>
<<Include>>
<<Include>>
<<Include>>
UC6: Deliver document via e-mail
UC15: Download document via unique link
UC16: Register to personal document store
UC8: Deliver document via postal mail
UC9: Deliver invoice via Zoomit
UC10: Confirm document delivery (Zoomit)
UC18: Register customer organization
UC19: Unregister customer organization
UC7: Notify of e-mail delivery failure
<<E-mail Provider>>
Unregistered ...
Print & Postal Service
Zoomit Service
DocProc Customer Administrator
E-mail Provider

Applied to DocProc

UC3 - Initiate document processing

1. The Customer Organization indicates that he/she/it wants to initiate a batch of document processing jobs.
2. The Customer Organization indicates the type of the documents to be generated (i.e., payslip or invoice).
3. If this non-recurrent batch should be handled with a different priority than the default priority of this tustomer organization, the Customer Organization indicates this priority to the System.
4. The Customer Organization provides the raw data to the System in an appropriate format (e.g., an Excel file, a zipped XML file, a CSV file).
5. The System validates the received raw data (e.g., checking whether the content of the received Excel file can be read or whether a received XML file is correctly formatted).
6. If the Customer Organization indicated that this raw data corresponds to a recurring batch of document processing jobs, the System validates that the raw data does not contain more than the maximally allowed number of documents to be generated.
7. The System validates the individual entries in the raw data (e.g., whether an e-mail address is present for every document that should be sent using e-mail or whether the city and the postal code in the postal addresses are consistent).

Which steps involve crossing the context boundary?

Applied to PMS

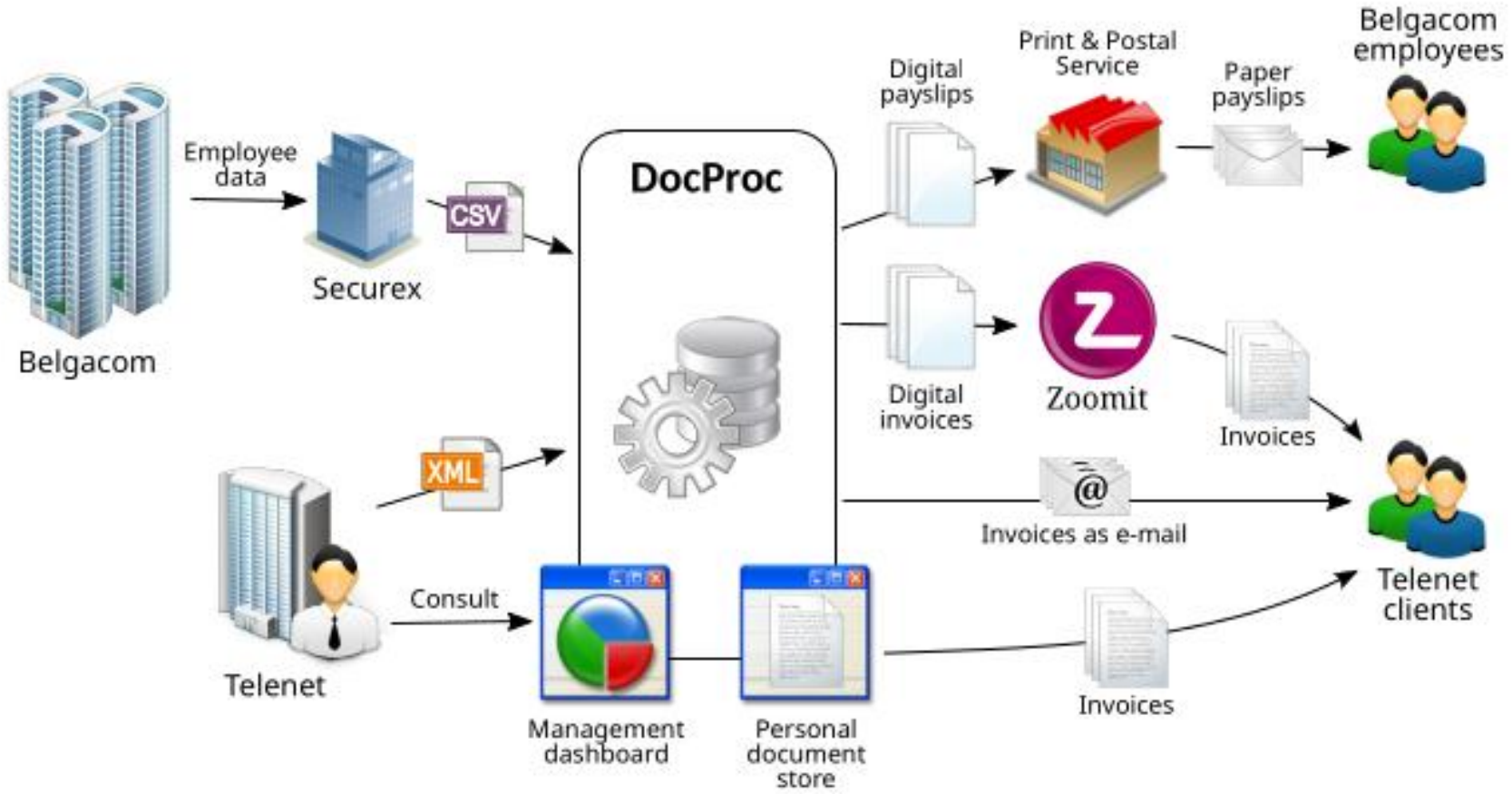
UC7 : Notify of e-mail delivery failure

Main scenario:

1. The E-mail Provider indicates that the delivery of a certain e-mail has failed, e.g., because the e-mail address does not exist or does not exist any more.
2. The System looks up the corresponding customer organization and notifies the appropriate Customer Administrator of this customer organization of the failure (include: UC22 : Notify customer administrator).
3. The System marks that the e-mail delivery has failed and adds a description of the error.

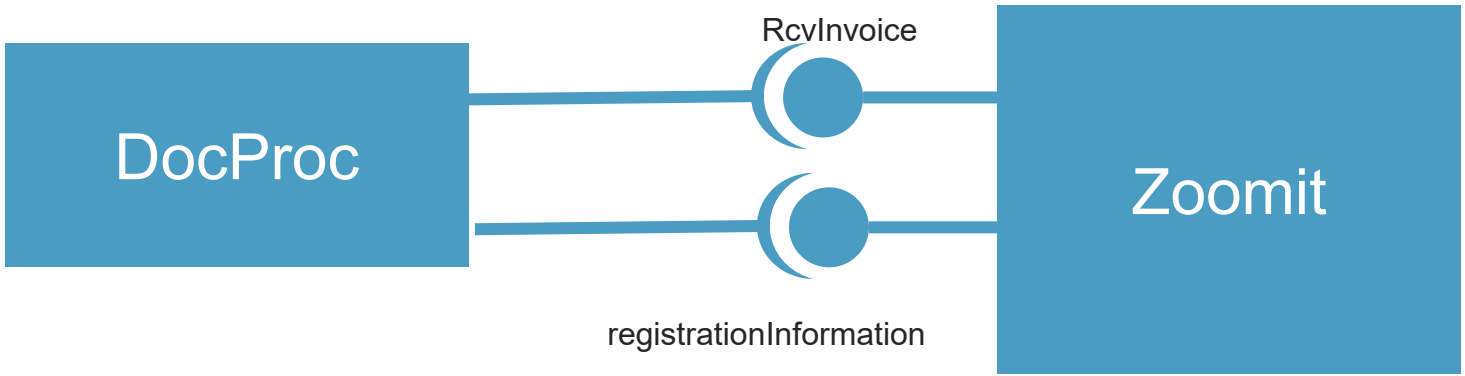
Which steps involve crossing the context boundary?

Overview and positioning

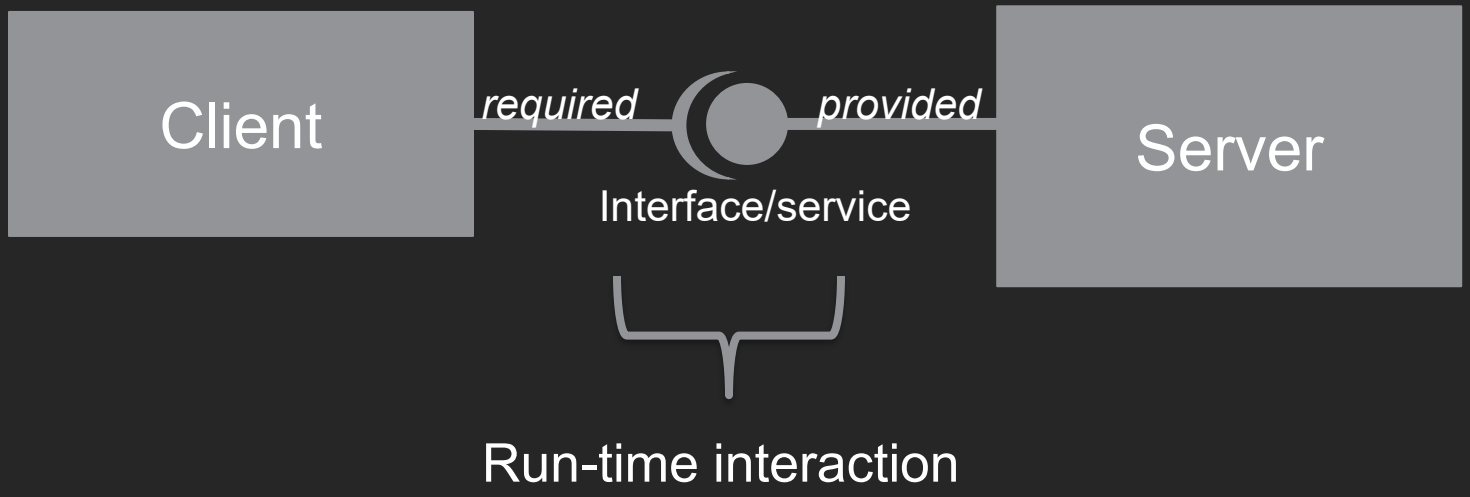


What about DocProc context?

One piece of the context puzzle is provided to you

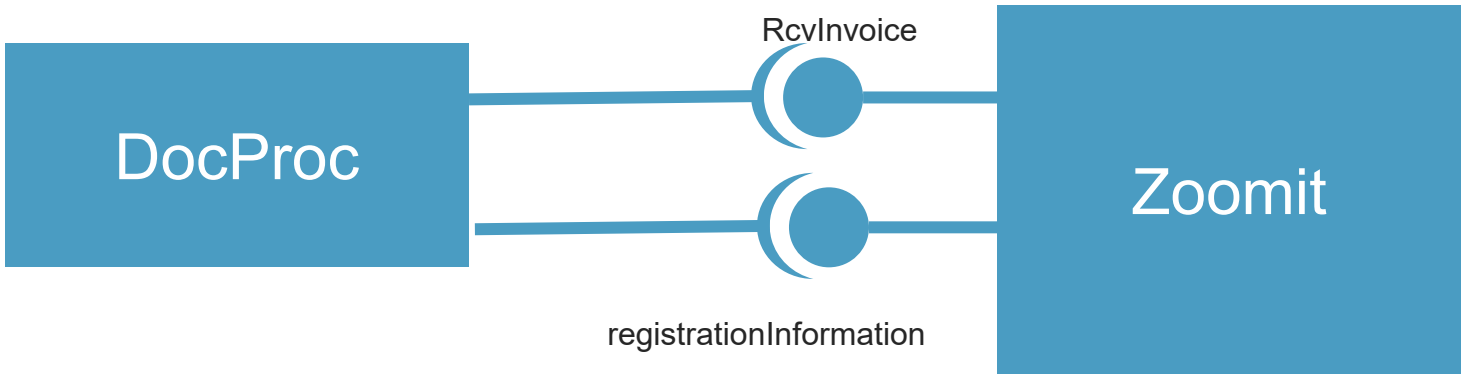


Intermezzo: preview on UML component/interface notation



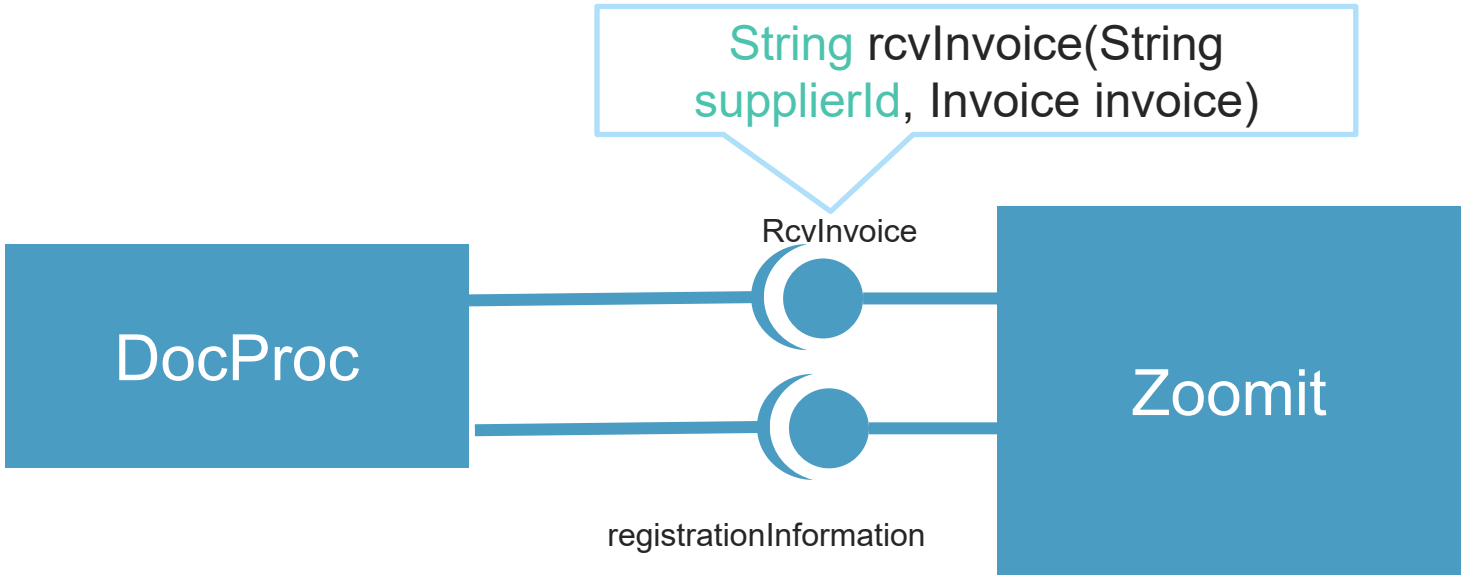
What about DocProc context?

One piece of the context puzzle is provided to you



What about DocProc context?

One piece of the context puzzle is provided to you



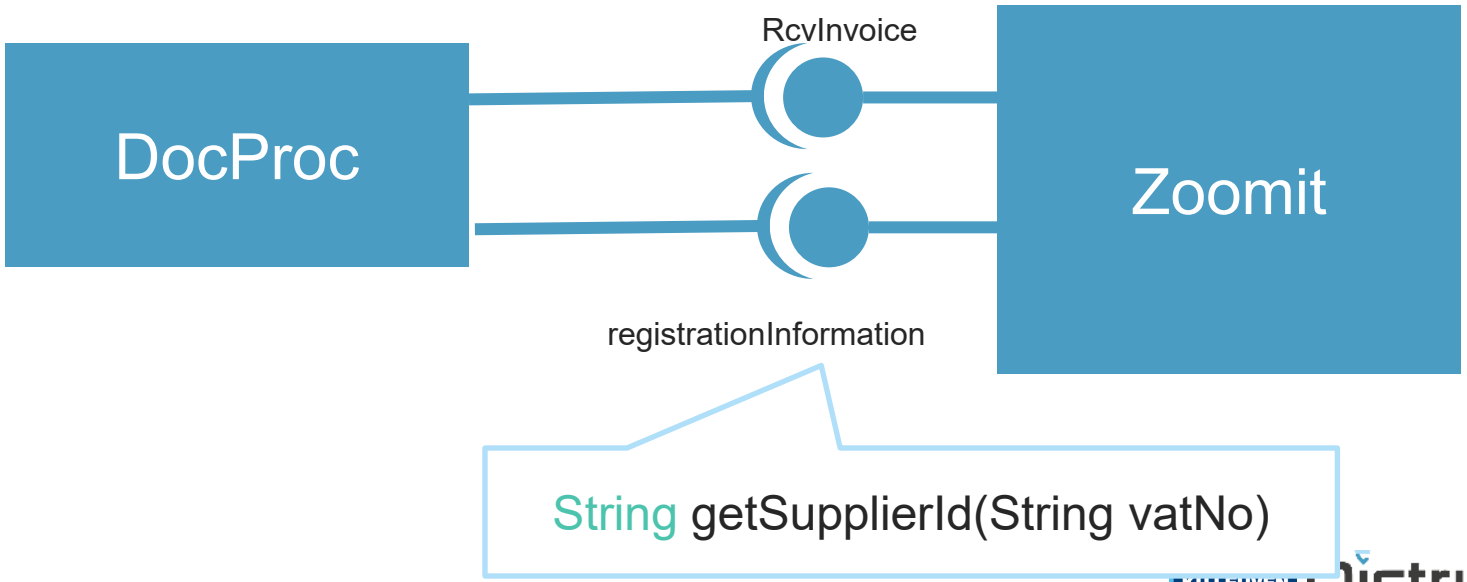
Introducing some real-world complexity

- String rcvInvoice(String supplierId, Invoice invoice)
  - **Effect**: Zoomit delivers the received invoice to the the inbox of the correct customer and notifies him or her a new invoice from the company identified by supplierId is available. A string representing a unique identifier for this invoice within Zoomit is returned to the caller.

Zoomit-internal supplierId - is different from ours/our customers naming scheme (VAT nb)

What about DocProc context?

One piece of the context puzzle is provided to you



Introducing some real-world complexity

- String getSupplierId(String vatNo)
  - **Effect**: Returns the unique identifier assigned by Zoomit to identify the company matching the given VAT identification number. Zoomit assigns this on registration of a company with their service.

Zoomit-internal supplierId - is different from ours/our customers naming scheme (VAT nb)

Several simplifications

- Simplified (abstracted) API for working with external parties (appendix B)
  - Zoomit & P&P Service
- But realistically, these APIs are considered immutable - to us, they are a **constraint**

- (cf. Twin Peaks lecture - some else's decisions are my requirements)

What do you think?

is a system's context generally **fixed and immutable**?

What about DocProc ?

- Think about:
  - External systems versus humans
  - How does interaction with third parties happen?
  - How/where is the system provisioned/deployed?
  - Are we using third-party libraries?

we know (**constraint**)

we assume (**uncertainty**)

will still have to decide (**architectural decision has to be made later**)

is a system's context **fixed and immutable**?

Use cases, ASRs, Quality attribute scenarios are dominantly **prescriptive** **modeling techniques**

But **systems are not developed in isolation**;
awareness of and attention to **context is important**

Awareness of and attention to context is important

- **We know:**
  - APIs or standards may evolve
  - different API may be dictated by third parties
- **We assume**:
  - we may be wrong; third party can make different assumptions;
  - assumption may hold now but not later
- **We still have to decide**:
  - anticipate degrees of freedom
  - not all future decisions will be up to us;

Summary – take home

System Context

- Shows your understanding of the system under design in relation to its environment

  later, this translates into CONTEXT DIAGRAMS
- Affects **both** requirements & architecture
  - Not immutable: architectural decisions, assumptions, external factors will influence context
  - No greenfield development: there are existing constraints (external parties, technology)
- **Quality attributes** are influenced by context
  - Usability, interoperability, security, …
- Context influences **semantics** of what we model (cf. context diagrams - later)